Computer Architecture
CS 622 / EE 520
Spring 2026

Lecture 14

Shahid Masud

Topics

- Multi-issue and VLIW
- Some solved Examples
  - Loops / Unrolling
  - VLIW
  - Speculative Execution

Quiz 3
Today (Wed)

Midterm
26 Oct

Assignment
2 Due 9 Nov

Further Readings

Reading Notes

Readings:
Chapter 3 of course textbook

Multiple Issue Concept - Superscalar

Since the execute stage of the pipeline is really a bunch of different *functional units*, each doing its own task, it seems tempting to try to execute multiple instructions *in parallel*, each in its own functional unit. To do this, the fetch and decode/dispatch stages must be enhanced so they can decode multiple instructions in parallel and send them out to the "execution resources"...

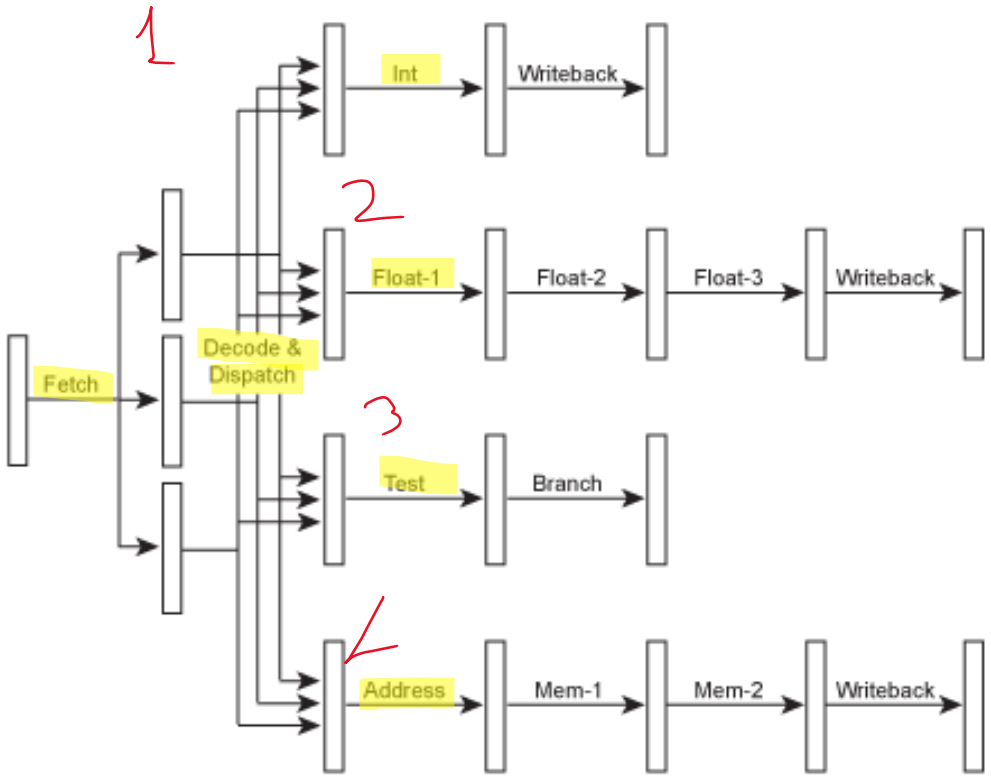


Figure 7 – A superscalar microarchitecture.

Of course, now that there are independent pipelines for each functional unit, they can even have different numbers of stages. This allows the simpler instructions to complete more quickly, reducing *latency* (which we'll get to soon). Since such processors have many different pipeline depths, it's normal to refer to the depth of a processor's pipeline when executing *integer* instructions, which is usually the shortest of the possible pipeline paths, with the memory and floating-point pipelines implied as having a few additional stages. Thus, a processor with a "10-stage pipeline" would use 10 stages for executing integer instructions, perhaps 12 or 13 stages for memory instructions, and maybe 14 or 15 stages for floating-point. There are also a bunch of bypasses within and between the various pipelines, but these have been left out of the diagram for simplicity.

VLIW Details

Basic VLIW Approach

The Basic VLIW Approach

VLIWs use multiple, independent functional units. Rather than attempting to issue multiple, independent instructions to the units, a VLIW packages the multiple operations into one very long instruction or requires that the instructions in the issue packet satisfy the same constraints. Because there is no fundamental difference in the two approaches, we will just assume that multiple operations are placed in one instruction, as in the original VLIW approach.

Because the advantage of a VLIW increases as the maximum issue rate grows, we focus on a wider issue processor. Indeed, for simple two-issue processors, the overhead of a superscalar is probably minimal. Many designers would probably argue that a four-issue processor has manageable overhead, but as we will see later in this chapter, the growth in overhead is a major factor limiting wider issue processors.

Very Long Instruction Word

- Each "instruction" has explicit coding for multiple operations
  - In IA-64, grouping called a "packet"
  - In Transmeta, grouping called a "molecule" (with "atoms" as ops)
- Tradeoff instruction space for simple decoding
  - The long instruction word has room for many operations
  - By definition, all the operations the compiler puts in the long instruction word are independent => execute in parallel
  - E.g., 1 integer operation/branch, 2 FP ops, 2 Memory refs
    - » 16 to 24 bits per field => 5*16 or 80 bits to 5*24 or 120 bits wide
  - Need compiling technique that schedules across several branches

Parallelism needed for VLIW

Let's consider a VLIW processor with instructions that contain five operations, including one integer operation (which could also be a branch), two floating-point operations, and two memory references. The instruction would have a set of fields for each functional unit—perhaps 16–24 bits per unit, yielding an instruction length of between 80 and 120 bits. By comparison, the Intel Itanium 1 and 2 contain six operations per instruction packet (i.e., they allow concurrent issue of two three-instruction bundles, as Appendix H describes).

To keep the functional units busy, there must be enough parallelism in a code sequence to fill the available operation slots. This parallelism is uncovered by unrolling loops and scheduling the code within the single larger loop body. If the unrolling generates straight-line code, then *local scheduling* techniques, which operate on a single basic block, can be used. If finding and exploiting the parallelism require scheduling code across branches, a substantially more complex *global scheduling* algorithm must be used. Global scheduling algorithms are not only more complex in structure, but they also must deal with significantly more complicated trade-offs in optimization, because moving code across branches is expensive.

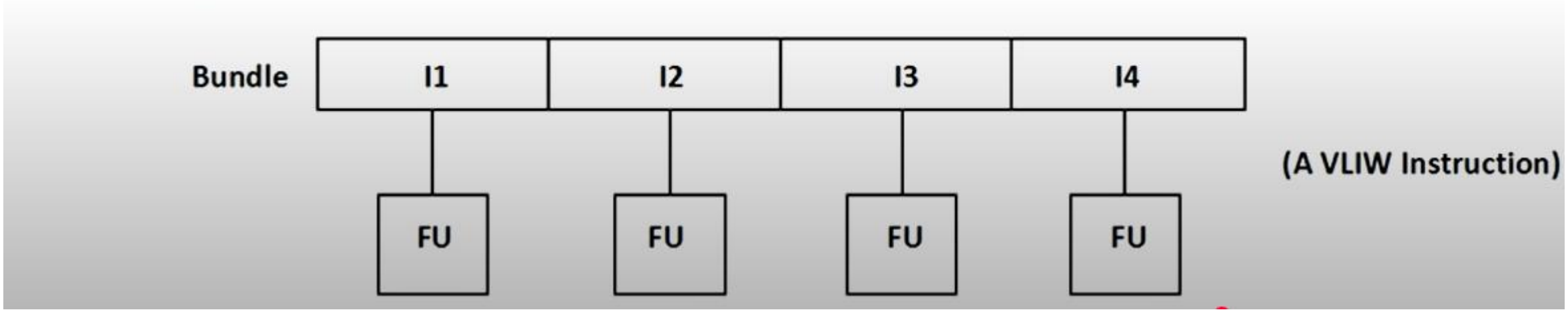
VLIW Processor

VLIW is also an architectural design that exploit instruction level parallelism.

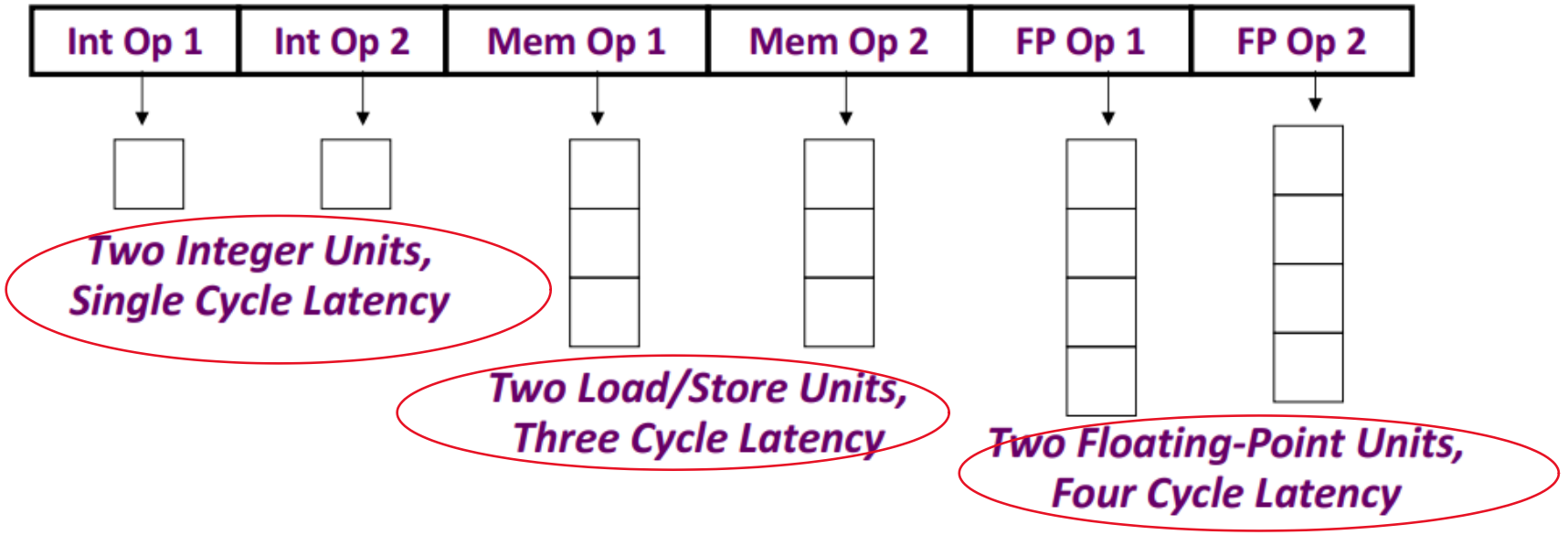
The VLIW processor rely on compile time analysis to identify and bundle together instructions that can be executed concurrently.

Compiler is responsible for static scheduling of instructions in VLIW processor. Compiler finds out which operations can be executed in parallel in the program and groups together these operations in single instruction known as a bundle or an issue packet which is the very large instruction word.Compiler ensures that an operation is not issued before its operands are ready.

They deploy multiple independent functional units.

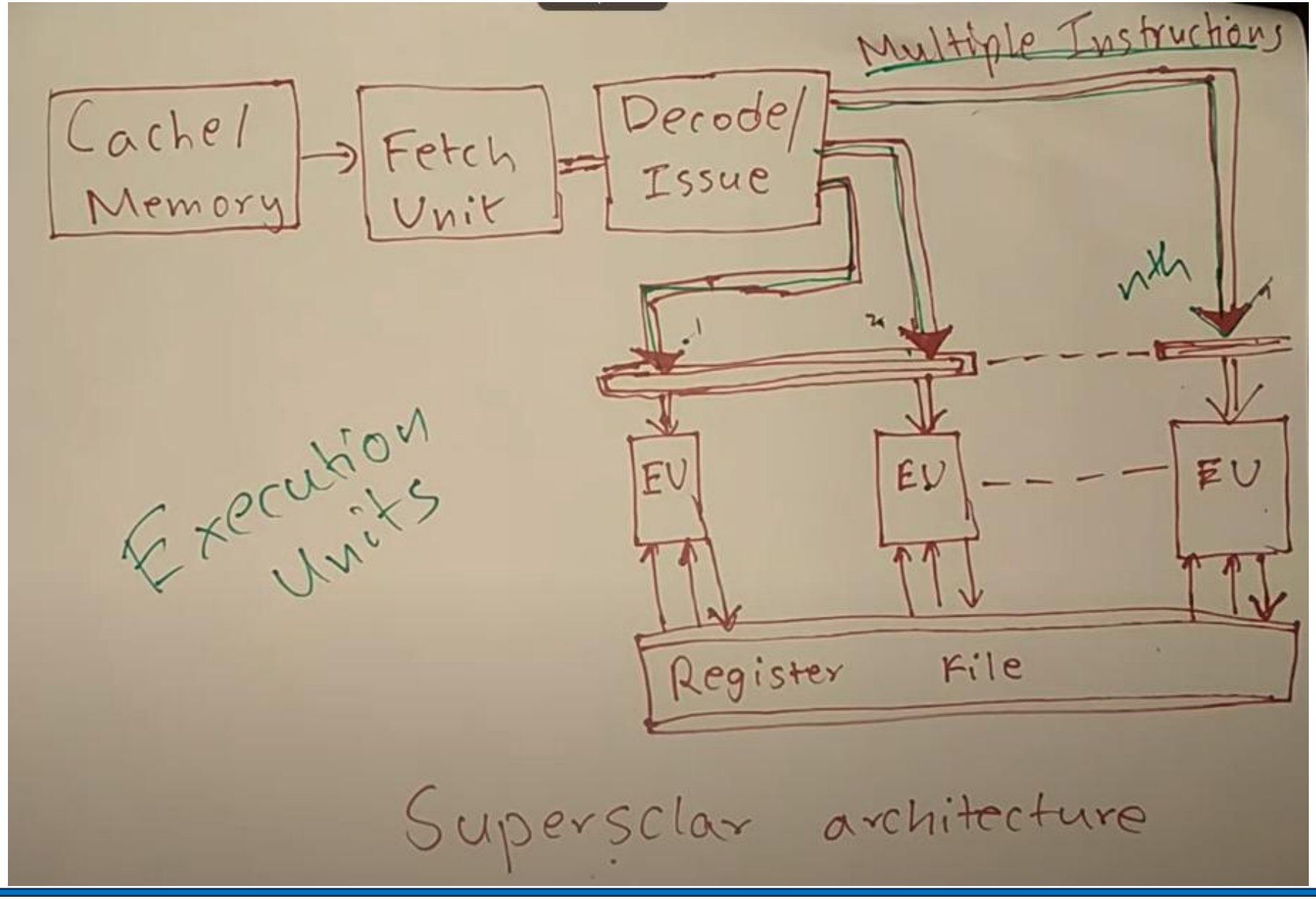


Form VLIW Instruction Word

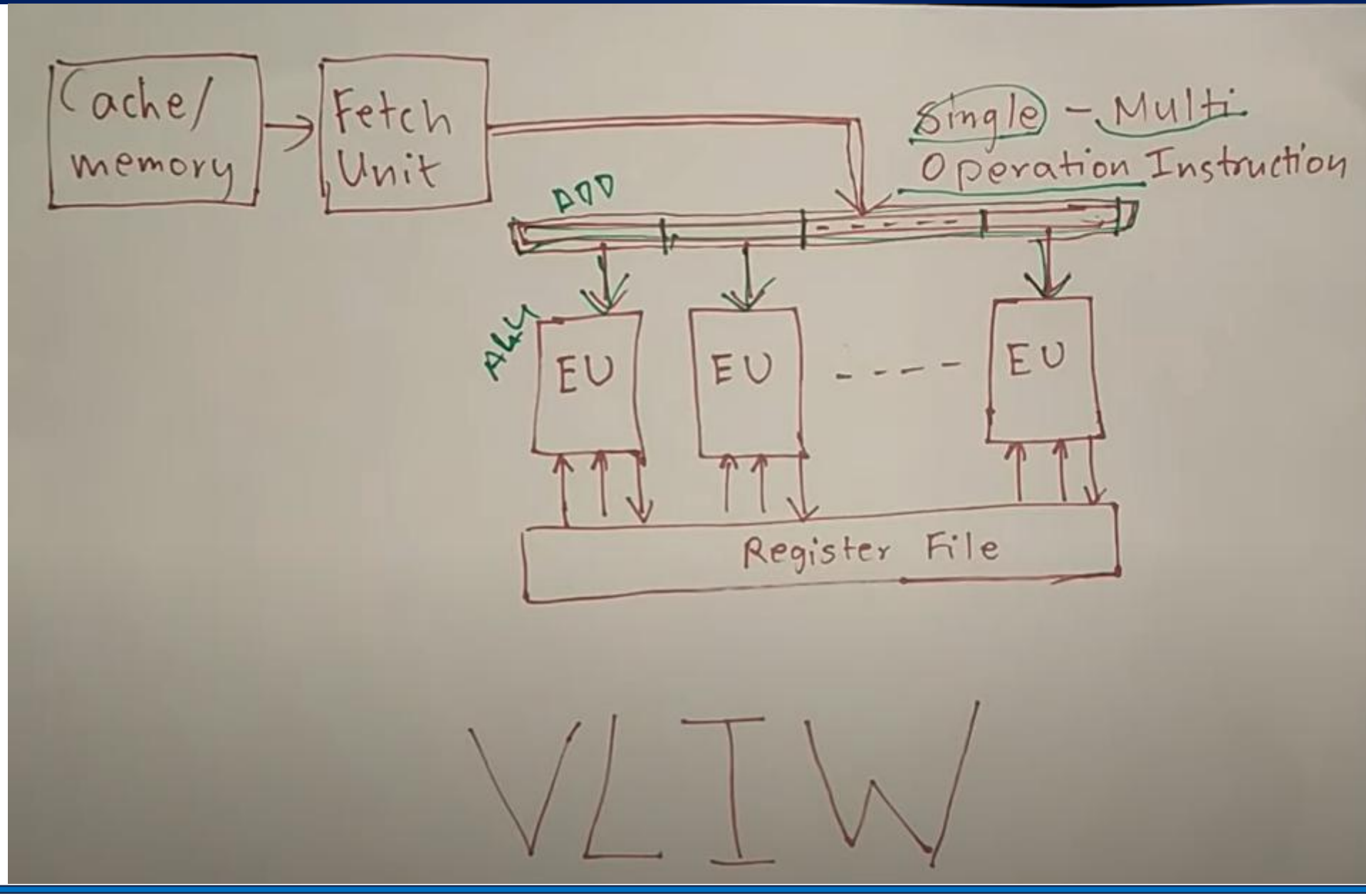


- Multiple operations packed into one instruction
- Each operation slot is for a fixed function
- Constant operation latencies are specified
- Architecture requires guarantee of:
  - Parallelism within an instruction => no cross-operation RAW check
  - No data use before data ready => no data interlocks

Superscalar Architecture



VLIW Architecture



Compare VLIW and Superscalar

VLIW Processors	Superscalar Processors
Parallelism is performed statically at compile time by the compiler.	Parallelism is performed dynamically at run time by hardware.
Data dependency is checked by compiler.	Data dependency is checked and resolved by hardware.
Less parallelism can be exploited.	More parallelism can be exploited.
Simple hardware and complex compiler.	Simple compiler and complex hardware.

Problems in VLIW Approach

For the original VLIW model, there were both technical and logistical problems that made the approach less efficient. The technical problems were the increase in code size and the limitations of the lockstep operation. Two different elements combine to increase code size substantially for a VLIW. First, generating enough operations in a straight-line code fragment requires ambitiously unrolling loops (as in earlier examples), thereby increasing code size. Second, whenever instructions are not full, the unused functional units translate to wasted bits in the instruction encoding. In Appendix H, we examine software scheduling approaches, such as software pipelining, that can achieve the benefits of unrolling without as much code expansion.

To combat this code size increase, clever encodings are sometimes used. For example, there may be only one large immediate field for use by any functional unit. Another technique is to compress the instructions in main memory and expand them when they are read into the cache or are decoded. In Appendix H, we show

Problems with VLIW Processors

- Large number of registers needed in order to keep all FUs active (to store operands and results).
- Large data transport capacity is needed between FUs and the register file and between register files and memory.
- High bandwidth between instruction cache and fetch unit.
  Example: one instruction with 7 operations, each 24 bits ⇒ 168 bits/instruction.
- Large code size, partially because unused operations ⇒ wasted bits in instruction word.
- Incompatibility of binary cod
  - For example:
    If for a new version of the processor additional FUs are introduced ⇒
    the number of operations possible to execute in parallel is increased ⇒
    the instruction word changes ⇒
    old binary code cannot be run on this processor.

Hazard Detection in VLIW, Code Compatibility

Early VLIWs operated in lockstep; there was no hazard-detection hardware at all. This structure dictated that a stall in any functional unit pipeline must cause the entire processor to stall because all the functional units had to be kept synchronized. Although a compiler might have been able to schedule the deterministic functional units to prevent stalls, predicting which data accesses would encounter a cache stall and scheduling them were very difficult to do. Thus caches needed to be blocking and causing *all* the functional units to stall. As the issue rate and number of memory references became large, this synchronization restriction became unacceptable. In more recent processors, the functional units operate more independently, and the compiler is used to avoid hazards at issue time, while hardware checks allow for unsynchronized execution once instructions are issued.

Binary code compatibility has also been a major logistical problem for general-purpose VLIWs or those that run third-party software. In a strict VLIW approach, the code sequence makes use of both the instruction set definition and the detailed pipeline structure, including both functional units and their latencies. Thus different numbers of functional units and unit latencies require different versions of the code. This requirement makes migrating between successive implementations, or between implementations with different issue widths, more difficult than it is for a superscalar design. Of course, obtaining improved performance from a new superscalar design may require recompilation. Nonetheless, the ability to run old binary files is a practical advantage for the superscalar approach. In the domain-specific

Questions and Answers

- Solved Examples from:
  - Loop Unrolling 1
  - Loop Unrolling 2
  - VLIW Eg 1
  - VLIW Eg 2
  - VLIW Eg 3

Quiz 3